

Transformer

2026 年 9 月 28 日

1 引言

如今主流的大语言模型几乎都以 Transformer 为骨架，最典型的的就是 GPT——全称 *Generative Pre-trained Transformer*，“生成式预训练 Transformer”，名字里最后一个词就是它。想弄明白现在的 LLM 在做什么，Transformer 是绕不开的第一站。市面上的讲解不少，但大多要么照着论文堆公式，要么停在最浅的比喻层面，于是我想自己写一篇，它的结构和计算流程从头到尾走一遍，也方便日后忘记时回头翻翻。

为了让计算能摊在纸上手算，例子里只用四个 token、四维表示这样的尺寸；完整问答示例换成八个 token、十二维表示。两者都是教学尺寸，不是原论文的模型配置；优化器调度、分词算法和推理加速等工程话题也不展开，先把最核心的计算链条接通。本文讨论的是 *Attention Is All You Need* 中的原始编码器—解码器 Transformer，采用图中“先 Add、后 Norm”的连接方式，侧重直观理解，省略了部分纯计算性的细节，并附上了笔者自己的理解，如有错误，还请指正；主要参考了原论文、《动手学深度学习》的相关章节与 PyTorch 文档。

2 序列任务与注意力

2.1 序列任务的困难

Transformer 最初是为机器翻译设计的，所以先从这类任务的难处说起。机器翻译、问答等任务都是从一个序列产生另一个序列：比如输入“谁是第一个登上月球的人？”，输出一个人名。这里光认识单个词还不够——“第一个”“月球”“人”要组合在一起，才约束住问题的含义。也就是说，模型必须让不同位置的信息发生交互。

一种自然的处理方式是顺序读取。循环神经网络（RNN）在第 i 个位置计算

$$\mathbf{h}_i = f_{\theta}(\mathbf{h}_{i-1}, \mathbf{x}_i). \quad (2.1)$$

这里 $\mathbf{h}_i$ 是读到当前位置时的内部状态。由于计算 $\mathbf{h}_i$ 需要先得到 $\mathbf{h}_{i-1}$ ，同一个序列内部存在串行依赖；相隔很远的位置之间，也往往需要经过多步状态传递。LSTM 等结构改善了记忆问题，但没有消除这种逐位置计算的依赖。

注意力提供了另一种办法：处理当前位置时，直接对允许访问的位置打分，再汇总其中的信息。它在 Transformer 之前就已经被用于神经机器翻译；Transformer 的关键并不是首次发明注意力，而是用注意力承担主要的序列信息交互，不再依赖原来的循环结构。

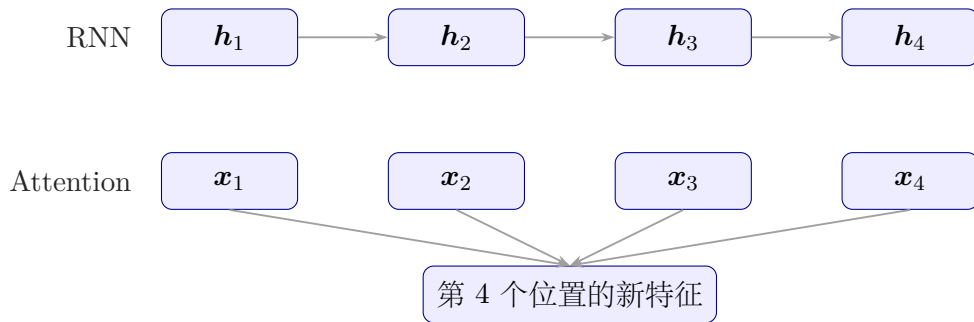


图 1: RNN 的串行依赖与注意力的直接汇总

需要说明的是，注意力并没有“根治”长距离依赖，Transformer 也不是在任何情况下都更快：完整注意力要考虑所有位置的配对，序列很长时计算量依然可观；层与层之间仍要顺序执行，生成答案时也还是一个 token

一个 token 地往外产生。它真正带来的，是一条更直接的信息通路，以及同一层内天然适合并行的计算方式。

2.2 输入与输出

原始 Transformer 采用 sequence-to-sequence (seq2seq) 形式：Encoder 把输入序列转换成一串连续向量；Decoder 读取这些向量以及已知的输出前缀，预测下一个 token。输入序列与输出序列不必一样长。

“seq2seq”描述输入、输出之间的组织方式，“注意力”描述内部如何汇总信息，两者并不冲突。所有向量默认按行摆放； n 表示输入长度， t 表示当前输出前缀长度， d 表示每个位置的表示维度。暂时不写 batch 维度，以免它遮住核心计算。

2.3 整体结构

在拆开每个部件之前，先看一眼全景（图 2）。原始 Transformer 由两座“塔”组成：左半 Encoder 从下往上是输入嵌入、位置编码和若干个相同的编码层；右半 Decoder 结构类似，但每个层里多出一个接收 Encoder 输出的子模块。现在看不懂细节没关系——底部圆圈与“+”表示加入位置编码，绕过各子模块的连线是残差相加，这些后面都会逐个拆开。把这张图记在心里，接下来沿着数据流动的顺序，从进入模型的第一步说起：一串文字是怎么变成向量的。

3 输入表示

3.1 Token 与词嵌入

计算机没法直接处理文字，第一步是把文字拆成基本单位并编号。Token 就是这样的基本单位——由分词器切分得到，未必是完整单词，也可能是字或子词。为方便手算，这里把 I love deep learning 当作四个 token。词表为它们分配编号；编号只负责索引，不表示语义大小。

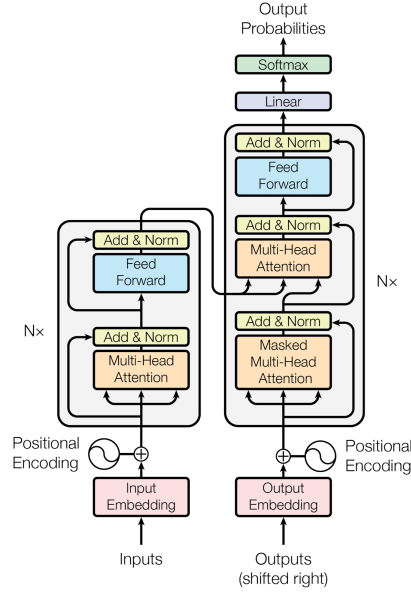


图 2: 原始 Transformer 结构 (原论文 Figure 1)

Embedding (嵌入) 是一张可训练的查找表。设词表为 $\mathcal{V}$, 则嵌入表

$$\mathbf{B} \in \mathbb{R}^{|\mathcal{V}| \times d} \quad (3.1)$$

为每个 token 存放一个 d 维向量。输入一个编号, 就取出对应的一行; 这不是把编号本身当成数值特征参与语义计算。

例如, 假设查表结果如表 1 所示。所有数字都是人为选择的演示值, 不代表真实模型中的词义。

表 1: 四个 token 的示意嵌入

Token	编号	查表得到的向量
I	12	[1, 0, 1, 0]
love	37	[0, 1, 1, 0]
deep	81	[0, 2, 0, 1]
learning	96	[1, 1, 0, 1]

按句子顺序把取出的四行叠起来，就得到当前句子的嵌入矩阵

$$\mathbf{E}_s = \begin{bmatrix} 1 & 0 & 1 & 0 \\ 0 & 1 & 1 & 0 \\ 0 & 2 & 0 & 1 \\ 1 & 1 & 0 & 1 \end{bmatrix} \in \mathbb{R}^{4 \times 4}. \quad (3.2)$$

注意区分： $\mathbf{B}$ 是整张词表的参数； $\mathbf{E}_s$ 是当前这句话查表得到的中间结果。一个位置对应一行，一个特征对应一列。

这些向量并不是人工定义好的“词义坐标”。从头训练时可以随机初始化，随后通过最终任务的损失与其他参数一起调整。每个维度通常没有预先规定的人类语义。参数固定时，同一个 token 的初始查表结果相同；经过后面的上下文计算后，它在不同句子中的表示才可以不同。

3.2 位置编码

有了词的向量，还缺顺序信息。“人咬狗”和“狗咬人”用的是同一组词，含义却完全相反。而后面的自注意力按内容汇总信息，本身并不感知“谁在前谁在后”，所以顺序必须显式地加进向量里。

原始 Transformer 生成一个与嵌入矩阵同形状的位置矩阵 $\mathbf{P}$ ，再相加。严格写出原论文中的嵌入缩放，有

$$\mathbf{X}^{(0)} = \sqrt{d} \mathbf{E}_s + \mathbf{P}, \quad \mathbf{E}_s, \mathbf{P}, \mathbf{X}^{(0)} \in \mathbb{R}^{n \times d}. \quad (3.3)$$

例如，八个 token、每个十二维时，相加前后都是 8×12 ，不是 8×13 。两个矩阵是在相同位置上逐元素相加，不是首尾拼接。

原始的正弦位置编码为

$$\begin{aligned} P_{p,2j} &= \sin\left(\frac{p}{10000^{2j/d}}\right), \\ P_{p,2j+1} &= \cos\left(\frac{p}{10000^{2j/d}}\right), \end{aligned} \quad (3.4)$$

其中位置 p 从零开始；对偶数 d ，有 $j = 0, \dots, d/2 - 1$ 。不同维度使用不同频率，因此一个位置对应一整组有规律的数，而不是单独附加一个整数编

号。至于为什么偏偏用正弦和余弦：原文的一个动机是这组波形有个好性质——对任意固定偏移 k ，位置 $p+k$ 的编码都能写成位置 p 编码的线性函数，方便模型学到相对位置关系。这里不必深究，把它理解成“每个位置对应一串固定规律的数”就够了。

3.3 相加不会混淆吗？

到这里，一个疑问很自然地冒出来：内容和位置是直接加在一起的，会不会有两个不同的“token + 位置”组合，加出同一个向量，让模型无从分辨？

如果这些数字是任意选的，确实可能。把式 (3.3) 的共同缩放暂时吸收入 $\mathbf{e}$ 的记号，取

$$\begin{aligned}\mathbf{e}(a) &= [0.1, 0.2, 0], & \mathbf{e}(b) &= [0, 0.1, -0.1], \\ \mathbf{p}_1 &= [0.1, 0.1, 0.1], & \mathbf{p}_2 &= [0, 0, 0],\end{aligned}$$

就有

$$\mathbf{e}(a) + \mathbf{p}_2 = \mathbf{e}(b) + \mathbf{p}_1 = [0.1, 0.2, 0]. \quad (3.5)$$

单看加出来的结果，区分不出它来自哪个组合。

那 Transformer 为什么还敢直接相加？因为相加的双方并不是随机的数字：位置编码是按固定规律生成的波形（式 (3.4)），嵌入是从训练中学出来的参数。模型从训练一开始就在“内容 + 位置”的联合表示上工作，完全可以学出适合任务的组合方式，并不需要先单独还原出“纯净的词义”、再单独还原出位置。式 (3.5) 说明的只是这种写法没有数学上的唯一性保证，而不是说相加一定会丢失信息。

到这里，输入侧的准备就完成了：每个 token 都带着内容与位置信息，变成了一行向量。接下来进入 Transformer 最核心的部分——注意力。

4 自注意力：从单头到多头

4.1 注意力层的输出

先澄清一个容易产生的直觉偏差：注意力并不是从句子里挑出一个“最重要的词”。它为每个查询位置分别生成一份上下文汇总——第 i 个位置有自己的分配比例，决定从各个位置分别取多少信息，而且这份比例随输入内容改变。

为了能把整个计算搬到纸上手算，下面直接给注意力子层指定一个输入

$$\mathbf{X} = \begin{bmatrix} 1 & 0 & 1 & 0 \\ 0 & 1 & 1 & 0 \\ 0 & 2 & 0 & 1 \\ 1 & 1 & 0 & 1 \end{bmatrix}. \quad (4.1)$$

四行仍对应 I/love/deep/learning。它的数值恰好沿用式 (3.2) 的嵌入矩阵，但请不要把它当成“嵌入加上位置编码”之后的结果——手算部分不再叠加位置编码，也省略投影偏置，专门用一个干净的输入把流程走通。

4.2 Query、Key、Value 的来源

同一个位置要参与两类工作：一类是决定“如何匹配和分配”，另一类是提供“真正被汇总的内容”。Query 与 Key 参与匹配，Value 参与最后的汇总。三者由输入经过不同的可训练线性投影得到：

$$\mathbf{Q} = \mathbf{X}\mathbf{W}_Q, \quad \mathbf{K} = \mathbf{X}\mathbf{W}_K, \quad \mathbf{V} = \mathbf{X}\mathbf{W}_V. \quad (4.2)$$

这里 $\mathbf{X}$ 不是 $\mathbf{Q}$ ，而是产生 $\mathbf{Q}, \mathbf{K}, \mathbf{V}$ 的原材料。

假设选择下面三套 4×2 的权重：

$$\mathbf{W}_Q = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 0 & 1 \\ 0 & -1 \end{bmatrix}, \quad \mathbf{W}_K = \begin{bmatrix} 0 & 0 \\ 1 & 0 \\ 0 & 1 \\ 0 & 0 \end{bmatrix}, \quad \mathbf{W}_V = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 0 \\ 0 & 1 \end{bmatrix}. \quad (4.3)$$

“乘权重矩阵”就是按系数组合原来的特征。对于 $\mathbf{x} = [a, b, c, d]$ ，这三个矩阵分别实现

$$\mathbf{q} = [a, b + c - d], \quad \mathbf{k} = [b, c], \quad \mathbf{v} = [a + c, b + d]. \quad (4.4)$$

真实模型不会固定使用这几条规则，而是在训练中调整系数。对所有行应用同一套规则，得到

$$\mathbf{Q} = \begin{bmatrix} 1 & 1 \\ 0 & 2 \\ 0 & 1 \\ 1 & 0 \end{bmatrix}, \quad \mathbf{K} = \begin{bmatrix} 0 & 1 \\ 1 & 1 \\ 2 & 0 \\ 1 & 0 \end{bmatrix}, \quad \mathbf{V} = \begin{bmatrix} 2 & 0 \\ 1 & 1 \\ 0 & 3 \\ 1 & 2 \end{bmatrix}. \quad (4.5)$$

三者形状都是 4×2 。到这里还没有交换 token 之间的信息，只是每一行分别做了特征变换。

4.3 分数矩阵

先只看第四个位置 **learning**。它的 Query 是 $\mathbf{q}_4 = [1, 0]$ ，与各 Key 点积：

$$\begin{aligned} \mathbf{q}_4 \mathbf{k}_1^\top &= 0, & \mathbf{q}_4 \mathbf{k}_2^\top &= 1, \\ \mathbf{q}_4 \mathbf{k}_3^\top &= 2, & \mathbf{q}_4 \mathbf{k}_4^\top &= 1. \end{aligned} \quad (4.6)$$

得到 $[0, 1, 2, 1]$ ，即 **learning** 对四个来源的匹配分数。注意分数里包含它自己——自注意力允许位置查看自己，但自己的分数并不保证最高：这里 **learning** 与 **deep** 的匹配分数（2）就高于与自己（1）。

其他查询位置也进行相同计算。将它们集中成矩阵乘法，就是

$$\mathbf{S} = \mathbf{QK}^\top = \begin{bmatrix} 1 & 2 & 2 & 1 \\ 2 & 2 & 0 & 0 \\ 1 & 1 & 0 & 0 \\ 0 & 1 & 2 & 1 \end{bmatrix}. \quad (4.7)$$

转置的作用是把每个 Key 从一行变成一列，使矩阵乘法实现所有 Query 与所有 Key 的点积：

$$(4 \times 2)(2 \times 4) \longrightarrow 4 \times 4.$$

因此 S_{ij} 的行 i 是接收信息的位置，列 j 是候选来源。 $\mathbf{S}$ 的列已经不是原始特征维度，而是序列中的位置。

4.4 缩放与 softmax

这个头的 Query 与 Key 都是二维，所以 $d_k = 2$ 。先把每个分数除以 $\sqrt{2}$ ；矩阵形状不变。第四行变成

$$[0, 1, 2, 1]/\sqrt{2} \approx [0, 0.707, 1.414, 0.707].$$

为什么要缩放？在一个简化假设下，Query 和 Key 的各分量独立、均值为零、方差为一，则

$$\text{Var}\left(\sum_{r=1}^{d_k} q_r k_r\right) = d_k. \quad (4.8)$$

点积的标准差随 $\sqrt{d_k}$ 增长——维度一高，分数动辄拉开很大差距，softmax 就会过度集中在少数位置上。提前除以 $\sqrt{d_k}$ ，相当于把这个尺度拉回来。当然，训练后的分量未必真满足上面的独立假设， $\sqrt{d_k}$ 更像一个稳妥的工程选择。

接下来对每一行分别做 softmax：

$$A_{ij} = \frac{\exp(S_{ij}/\sqrt{d_k})}{\sum_{r=1}^n \exp(S_{ir}/\sqrt{d_k})}. \quad (4.9)$$

第四行先取指数约为 $[1, 2.028, 4.113, 2.028]$ ，再除以这一行的总和，得到

$$\mathbf{a}_4 \approx [0.109, 0.221, 0.449, 0.221]. \quad (4.10)$$

四个数相加为一，表示当前查询位置如何分配汇总比例。所有行一起算得

$$\mathbf{A} = \text{softmax}(\mathbf{S}/\sqrt{2}) \approx \begin{bmatrix} 0.165 & 0.335 & 0.335 & 0.165 \\ 0.402 & 0.402 & 0.098 & 0.098 \\ 0.335 & 0.335 & 0.165 & 0.165 \\ 0.109 & 0.221 & 0.449 & 0.221 \end{bmatrix}. \quad (4.11)$$

$\mathbf{A}$ 叫作注意力权重矩阵，每一行是一种分配比例。它随输入内容和参数变化——同一句话换掉一个词，权重可能就是另一番样子。矩阵也不必对称：第 i 行第 j 列说的是“位置 i 从 j 取多少”，第 j 行第 i 列说的是反向的取法，本来就是两件事。

4.5 加权求和输出

第四个位置的输出是四个 Value 的加权和：

$$\begin{aligned} z_4 &= \sum_{j=1}^4 A_{4j} v_j \\ &\approx 0.109[2, 0] + 0.221[1, 1] + 0.449[0, 3] + 0.221[1, 2] \\ &\approx [0.660, 2.009]. \end{aligned} \quad (4.12)$$

以第一维为例： $0.109 \times 2 + 0.221 \times 1 + 0.449 \times 0 + 0.221 \times 1 \approx 0.66$ 。式中的权重只保留了三位小数，最终结果按未舍入的权重计算。

这和平均数很像，但不是每个来源都占 $1/4$ ，而是每个查询位置根据内容决定自己的权重。对所有位置一次完成汇总，就是

$$\mathbf{Z} = \mathbf{A}\mathbf{V} \approx \begin{bmatrix} 0.830 & 1.670 \\ 1.304 & 0.891 \\ 1.170 & 1.160 \\ 0.660 & 2.009 \end{bmatrix} \in \mathbb{R}^{4 \times 2}. \quad (4.13)$$

$\mathbf{A}$ 是“怎么取”的规则， $\mathbf{Z}$ 是“实际取到了什么”的特征。前者每行四个权重，后者每行两个输出特征，不能混为一谈。

现在把三步连起来，得到熟悉的公式：

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right) \mathbf{V}. \quad (4.14)$$

Scaled Dot-Product Attention

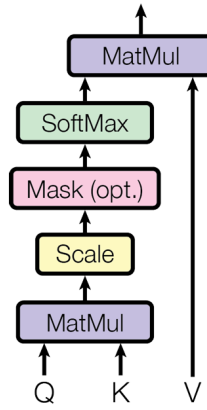


图 3: 缩放点积注意力 (原论文 Figure 2 左)

整个公式其实就是三步： $\mathbf{Q}\mathbf{K}^\top$ 完成匹配，softmax 把分数变成分配比例，再按比例汇总 $\mathbf{V}$ 的内容。后面要讲的多头注意力、交叉注意力，变的都是这三步里各矩阵的来源，套路本身不再变。

4.6 多头注意力

恢复到 $\mathbf{X} \in \mathbb{R}^{8 \times 12}$ 的教学尺寸。一个头只有一套注意力分配方式：对某个来源给出权重 a ，就会把该来源的整个 Value 向量乘上同一个 a 。多头允许同时计算多套分配与内容表示，再合并结果。

设有三个头，每个头的 Query、Key、Value 都是四维。第 h 个头计算

$$\mathbf{Q}_h = \mathbf{X}\mathbf{W}_Q^{(h)}, \quad \mathbf{K}_h = \mathbf{X}\mathbf{W}_K^{(h)}, \quad \mathbf{V}_h = \mathbf{X}\mathbf{W}_V^{(h)}, \quad (4.15)$$

其中每个权重矩阵都是 12×4 。因此每个头都能组合完整的十二维输入，并不是先截取原向量的前四维、中间四维和后四维。它们各自得到 8×8 的权重矩阵，以及 8×4 的输出 $\mathbf{Z}_h$ 。

需要说明的是，并不存在“一个头专门负责主谓搭配、一个头专门负责指代”这样的预设分工。各头只是参数不同，靠同一个损失联合训练；它们最终学出怎样的分工，是训练的结果，不是设计时的规定。

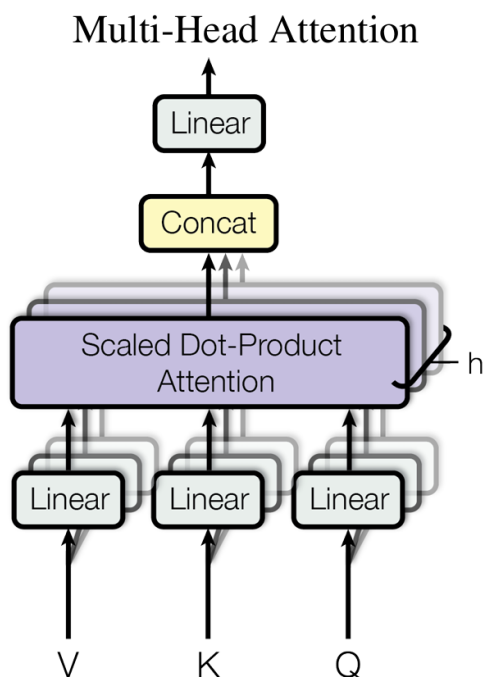


图 4: 多头注意力（原论文 Figure 2 右）

一般写成

$$\text{MHA}(\mathbf{X}) = \text{Concat}(\mathbf{Z}_1, \dots, \mathbf{Z}_H) \mathbf{W}_O, \quad \mathbf{W}_O \in \mathbb{R}^{Hd_v \times d}. \quad (4.16)$$

实现中可以把多个投影合成一次大矩阵乘法，再把投影后的结果拆成头。例如

$$\mathbf{X}[\mathbf{W}_Q^{(1)} \mid \mathbf{W}_Q^{(2)} \mid \mathbf{W}_Q^{(3)}] = [\mathbf{Q}_1 \mid \mathbf{Q}_2 \mid \mathbf{Q}_3].$$

所谓“拆头”常常发生在这一步，而不是在原始输入上直接切掉部分信息。

4.7 输出维度

各头的内部宽度并不必须是四。完全可以让每个头输出 8×5 ，三个头拼成 8×15 ，再选择

$$\mathbf{W}_O \in \mathbb{R}^{15 \times 12}, \quad (8 \times 15)(15 \times 12) \longrightarrow 8 \times 12. \quad (4.17)$$

回到十二维，是为了与网络主路径的宽度一致，方便残差相加和逐层连接。常见的 $d_v = d/H$ 只是一种让内部总宽度、计算量和参数量都比较可控的取法，并非硬性规定。

八行之所以保留，是因为这里有八个 Query，需要分别输出八个向量。行数由查询位置数决定，列数则由特征设计和输出投影决定。

至此，注意力的两种形态——单头与多头——都齐了。不过光有注意力还组不成一个完整的层：把它装进网络，还需要残差连接、归一化和前馈网络。

5 Add、Norm 与 FFN

5.1 残差连接

暂时不考虑归一化，用 F_θ 表示注意力或 FFN 这样的可训练模块。没有残差时，模块输出直接作为结果；有残差时，再加上该模块的输入：

$$\text{无残差: } \hat{\mathbf{y}} = F_\theta(\mathbf{x}), \quad \text{有残差: } \hat{\mathbf{y}} = \mathbf{x} + F_\theta(\mathbf{x}). \quad (5.1)$$

残差把输入分成两路：一路经过模块变换，一路原样直通，最后相加。注意，直通的那一路从训练的第一步就在起作用——损失始终在评判 $\mathbf{x} + F_\theta(\mathbf{x})$ 这整个整体，而不是先把 F_θ 单独训好、最后再把 $\mathbf{x}$ 补上去。

例如，我们希望实现 $T(x) = 3x$ ，分支取 $F_\theta(x) = \theta x$ 。没有残差时需要 $\theta = 3$ ；有残差时只需要 $\theta = 2$ ，因为 $x + 2x = 3x$ 。目标没有改变，达到目标所需的参数改变了。

取训练样本 $x = 2$ 、目标 $y^* = 6$ 。有残差时，真正参与损失的是

$$L(\theta) = (2 + 2\theta - 6)^2, \quad \frac{dL}{d\theta} = 4(2 + 2\theta - 6). \quad (5.2)$$

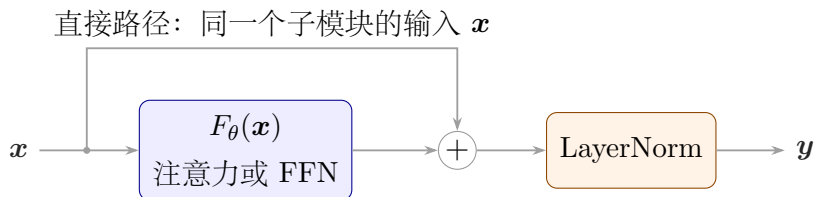


图 5: 残差连接与归一化

最优解是 $\theta = 2$: 分支输出 $2 \times 2 = 4$, 加上输入的 2 恰好等于 6。如果让分支直接去拟合目标 6, 加回输入反而得到 8。也就是说, 残差结构下分支要学的不是目标本身, 而是“距离目标还差多少”。

用 $T(\mathbf{x})$ 表示理想变换, 那么 $F_\theta(\mathbf{x})$ 学的就是差值 $T(\mathbf{x}) - \mathbf{x}$, 也就是“更新量”。不过要留意: 训练数据只给最终答案, 中间并没有一份现成的 $T(\mathbf{x})$ 摆在那里供相减, 网络是从整体损失中自己摸索出每层该输出什么更新的。

残差分支学的是“更新量”, 但更新量并不总是很小。

从 2 变到 6, 需要的更新就是 4, 不小也不该小; 只有某个子层确实不需要改变输入时, 零更新才合理。当然, “能学差值”只是代数上的可能性, 能不能学好仍取决于网络的表达能力和训练过程。

5.2 直接路径与反向传播

残差对训练的好处, 从反向传播的角度看得更清楚。先用一维说明: 设 $y = x + F_\theta(x)$, 后面的损失为 L , 链式法则给出

$$\frac{\partial L}{\partial x} = \frac{\partial L}{\partial y} (1 + F'_\theta(x)). \quad (5.3)$$

没有残差时, 括号里只有 $F'_\theta(x)$ 。假如这个导数是 0.01、后一层传来的梯度是 1, 两种情况分别得到 0.01 和 1.01——哪怕分支学得不好、导数很小, 梯度仍能沿着那个 +1 的直通路回到浅层。这不是“永不消失”的保证, 但确实给了梯度一条不依赖分支质量的退路。

多维情况对应 $\mathbf{I} + \mathbf{J}_F$, 其中 $\mathbf{J}_F$ 是分支的雅可比矩阵。原始 Transformer 后面还接 LayerNorm, 梯度也要经过它, 不能把整层视为完全不变的恒等通道。只把分支设成零, 原图的子层仍输出 $\text{LN}(\mathbf{x})$, 而不是严格的 $\mathbf{x}$ 。

5.3 LayerNorm 的归一化对象

先做

$$\mathbf{U} = \text{MHA}(\mathbf{X}), \quad \mathbf{Z} = \mathbf{X} + \mathbf{U}, \quad \mathbf{Y} = \text{LN}(\mathbf{Z}). \quad (5.4)$$

把前两项代入，就是常见的写法 $\mathbf{Y} = \text{LN}(\mathbf{X} + \text{MHA}(\mathbf{X}))$ 。写这么长只是为了看清结构——LN 实际接收的只有相加后的 $\mathbf{Z}$ 这一个矩阵。

对某一行 $\mathbf{z} = [z_1, \dots, z_d]$ ，计算

$$\mu = \frac{1}{d} \sum_{j=1}^d z_j, \quad \sigma^2 = \frac{1}{d} \sum_{j=1}^d (z_j - \mu)^2, \quad (5.5)$$

再逐维得到

$$\text{LN}(\mathbf{z})_j = \gamma_j \frac{z_j - \mu}{\sqrt{\sigma^2 + \epsilon}} + \beta_j. \quad (5.6)$$

ϵ 是避免数值问题的小常数； γ, β 是可训练的缩放与平移参数。每个 token 分别沿自己的特征维度计算统计量；它不把整句话混在一起平均，也不是沿 batch 计算。

例如 $\mathbf{z} = [1, 2, 3, 4]$ 的均值为 2.5、方差为 1.25。暂取 $\gamma = \mathbf{1}, \beta = \mathbf{0}$ 并忽略很小的 ϵ ，得到

$$\text{LN}(\mathbf{z}) \approx [-1.342, -0.447, 0.447, 1.342].$$

若把整个向量放大一百倍，换成 $[100, 200, 300, 400]$ ，标准化后的结果几乎不变——LN 关心的是各维度之间的相对起伏，而不是绝对尺度。当然它也只是调整尺度，并不创造信息；配合可训练的 γ 与 β ，最终输出的均值和方差也不必再是零和一。

尺度为什么值得管？忽略偏置，若输入整体放大 c 倍，投影的 $\mathbf{Q}, \mathbf{K}$ 也随之放大，点积变成 $c^2 \mathbf{Q} \mathbf{K}^\top$ ，softmax 的集中程度就可能明显改变。LN 有助于让后续层面对较受控的数值尺度，并改善深层网络的训练性质。

原图使用先相加、再归一化的 Post-LN。把 LN 放到分支前的 Pre-LN 是另一种结构，例如 $\mathbf{y} = \mathbf{x} + F_\theta(\text{LN}(\mathbf{x}))$ ；两者不是等价的括号挪动，训练性质也不同。本文只追踪原图，不在这里展开变体。

5.4 FFN 与非线性

注意力负责按内容汇总不同位置的信息；FFN 则对每个位置已经得到的特征继续做变换。原始结构为

$$\text{FFN}(\mathbf{x}) = \text{ReLU}(\mathbf{x}\mathbf{W}_1 + \mathbf{b}_1)\mathbf{W}_2 + \mathbf{b}_2. \quad (5.7)$$

例如可选 $12 \rightarrow 48 \rightarrow 12$ 的宽度，其中 $\text{ReLU}(x) = \max(x, 0)$ 是逐元素取正部。中间扩展提供更多候选特征，非线性使它不再只是一次固定的线性组合。同一层的各位置共用 FFN 参数，但分别计算。

一个极小的例子就能展示用意。让输入为一个数 x ，两次线性变换和一次 ReLU 实现

$$x \rightarrow [x, -x] \rightarrow [\max(x, 0), \max(-x, 0)] \rightarrow |x|. \quad (5.8)$$

输入 -2 ，依次得到 $[-2, 2]$ 、 $[0, 2]$ ，最后得到 2 。这正是一次 $1 \rightarrow 2 \rightarrow 1$ 的 FFN。

如果拿掉非线性、暂不写偏置，则

$$(\mathbf{x}\mathbf{W}_1)\mathbf{W}_2 = \mathbf{x}(\mathbf{W}_1\mathbf{W}_2),$$

两次线性变换合并成一次，包含偏置时也只是合并成一个仿射变换——中间维度升得再高，表达能力也不会变。升维的价值，要靠非线性来兑现。至于注意力里也有 softmax 这样的非线性，两者并不冲突：FFN 的角色是提供逐位置的非线性特征变换，在这一步不重新混合 token 之间的信息。

5.5 完整的 Encoder Block

忽略 dropout，原始 Encoder 一层只需写成

$$\begin{aligned} \mathbf{H} &= \text{LN}_1(\mathbf{X} + \text{MHA}(\mathbf{X})), \\ \mathbf{Y} &= \text{LN}_2(\mathbf{H} + \text{FFN}(\mathbf{H})). \end{aligned} \quad (5.9)$$

每个主要子模块都配一组 Add 与 Norm，所以一层有两组。第二次 Add 加回的是 FFN 的输入 $\mathbf{H}$ ，不是最初的 $\mathbf{X}$ 。两个 LN 有各自的可训练参数，但输出形状都不变。

原论文训练时还在分支输出等位置使用 dropout，更完整的第一行是 $\text{LN}_1(\mathbf{X} + \text{Dropout}(\text{MHA}(\mathbf{X})))$ 。本文为突出主线，后文一律省略它。

到这里，Encoder 的一层就完整了。把 N 块这样的结构堆起来，就得到整个 Encoder：

$$\mathbf{M} = \text{Encoder}(\mathbf{X}^{(0)}) \in \mathbb{R}^{n \times d}. \quad (5.10)$$

各层结构相同，但参数不同——后一层重新计算自己的 Query、Key、Value 和注意力，不是重复使用第一层的权重矩阵。 $\mathbf{M}$ 的每一行仍对应问题中的一个位置，只是已经受到上下文与多层特征变换的影响。

值得强调的是，Encoder 的输出是一整串向量，而不是把整句话压缩成的一个向量——更早的 seq2seq 系统正是后一种做法，句子一长，一个向量就装不下全部信息。Transformer 把这串表示原封不动地交给 Decoder，让它按需取用。下一节就看 Decoder 怎么接住它。

6 Decoder：掩码自注意力与交叉注意力

Encoder 侧的零件到这里都齐了。Decoder 的每一层用的也是这些零件，只有两处不同：自注意力带上了因果掩码，中间多出一个读取 $\mathbf{M}$ 的交叉注意力。回到图 2 的右半边，从下往上看。

6.1 掩码自注意力

第一次生成时，Decoder 的输入只有开始标记 $\langle \text{BOS} \rangle$ 。以后则是 $\langle \text{BOS} \rangle$ 加上已经生成的 token。图中右下角的 Output Embedding，表示把这串已知输出转成向量，不是把向量变成最终文字。

令当前前缀的表示为 $\mathbf{U} \in \mathbb{R}^{t \times d}$ 。在掩码自注意力中， Q, K, V 都来自 $\mathbf{U}$ ，但第 i 个位置只能访问 $j \leq i$ 的位置。用一个加在分数上的矩阵表示规

则：

$$C_{ij} = \begin{cases} 0, & j \leq i, \\ -\infty, & j > i. \end{cases} \quad \mathbf{A} = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}} + \mathbf{C}\right). \quad (6.1)$$

对于三个前缀 token, $\mathbf{C}$ 为

$$\begin{bmatrix} 0 & -\infty & -\infty \\ 0 & 0 & -\infty \\ 0 & 0 & 0 \end{bmatrix}.$$

因为 $\exp(-\infty) = 0$, 不允许访问的未来位置得到零权重。掩码不是把当前输入 token 自己隐藏起来, 而是阻止它提前读取后面的 token。

这一模块后面仍有 Add 与 Norm。掩码自注意力的结果已经融入答案前缀内部的信息, 但要根据问题作答, 还需要下一步。

6.2 交叉注意力

设右边经过前一子模块后得到 $\mathbf{H}$ 。交叉注意力对它和 Encoder 输出分别投影：

$$\mathbf{Q} = \mathbf{H}\mathbf{W}_Q^{\text{cross}}, \quad \mathbf{K} = \mathbf{M}\mathbf{W}_K^{\text{cross}}, \quad \mathbf{V} = \mathbf{M}\mathbf{W}_V^{\text{cross}}. \quad (6.2)$$

Query 来自当前的答案表示；Key、Value 来自问题表示。公式仍然是式 (4.14), 改变的是输入来源。

例如, 问题有八个 token, 答案前缀有三个 token, 每个头使用四维 Query、Key、Value。那么

$$\underbrace{\mathbf{Q}}_{3 \times 4} \underbrace{\mathbf{K}^\top}_{4 \times 8} \longrightarrow \underbrace{\mathbf{A}}_{3 \times 8}, \quad \underbrace{\mathbf{A}}_{3 \times 8} \underbrace{\mathbf{V}}_{8 \times 4} \longrightarrow \underbrace{\mathbf{Z}}_{3 \times 4}. \quad (6.3)$$

这表示答案的三个位置各自从问题的八个位置汇总信息。答案短、问题长并不冲突：Query 只决定每个答案位置“去问题里各取多少”，两边的长度本来就是相互独立的。

表 2: 三种注意力的连接关系（尺寸按单个头计）

模块	Query 的来源	Key、Value 的来源	权重矩阵
Encoder 自注意力	问题的当前表示	同一份问题表示	8×8
Decoder 掩码自注意力	答案前缀的当前表示	同一份前缀表示，屏蔽未来	3×3
Decoder 交叉注意力	右侧前一子模块的输出	Encoder 的最终输出	3×8

6.3 完整的 Decoder Block

忽略 dropout，一个 Decoder Block 完整写成

$$\begin{aligned}
 \mathbf{H} &= \text{LN}_1(\mathbf{U} + \text{MaskedMHA}(\mathbf{U})), \\
 \mathbf{R} &= \text{LN}_2(\mathbf{H} + \text{CrossMHA}(\mathbf{H}, \mathbf{M})), \\
 \mathbf{D} &= \text{LN}_3(\mathbf{R} + \text{FFN}(\mathbf{R})).
 \end{aligned} \tag{6.4}$$

三个主要子模块对应三组 Add 与 Norm，每一步都保持右侧 $t \times d$ 的形状。第二行加回的是右侧的 $\mathbf{H}$ ，不是左侧的 $\mathbf{M}$ 。

把 Decoder Block 和 Encoder Block 放在一起对比：自注意力换成了带因果掩码的版本，中间多出一个读取 $\mathbf{M}$ 的交叉注意力，其余的残差、LN、FFN 完全一致。下一层 Decoder 继续处理 $\mathbf{D}$ ，仍可访问同一份 $\mathbf{M}$ ；各层使用自己的投影参数。

7 逐步生成与训练

7.1 一个完整例子

结构讲完了，让整个模型完整跑一遍。假设任务是问答：输入“谁是第一个登上月球的人？”，期望模型简短地回答“阿姆斯特朗”（第一位登上月球的 Neil A. Armstrong）。当然，能答对的前提是接受过相关训练——只搭好结构、还没训练的网络，不会仅凭框架图就知道这条事实。

人为把问题分为八个 token:

[谁, 是, 第一个, 登上, 月球, 的, 人, ?].

这只是教学切分。Encoder 产生 $\mathbf{M} \in \mathbb{R}^{8 \times 12}$; 为了展示方便, 答案按单字逐个生成。

假设当前前缀是 “<BOS> 阿姆”, 它有三个 token。右边完成式 (6.4) 并经过全部 Decoder 层, 得到一个 3×12 的矩阵。每行都是对应位置的上下文表示。生成下一个 token 时, 取最后一行, 也就是当前 “姆” 所在位置的向量:

$$\mathbf{d}_{\text{last}} \in \mathbb{R}^{1 \times 12}. \quad (7.1)$$

取最后一行并不表示忽略前面位置; 这个向量已经通过注意力读取了允许访问的前缀和问题信息。

7.2 Linear 与 Softmax

假设目标词表有 10,000 个 token。输出线性层采用

$$\mathbf{W}_{\text{vocab}} \in \mathbb{R}^{12 \times 10000}, \quad \ell = \mathbf{d}_{\text{last}} \mathbf{W}_{\text{vocab}} + \mathbf{b} \in \mathbb{R}^{1 \times 10000}. \quad (7.2)$$

这里每个分数对应词表中的一个候选 token, 而不是一个输入位置。再计算

$$P(y_t = j \mid s, y_{<t}) = \frac{\exp(\ell_j)}{\sum_{r=1}^{|\mathcal{V}_{\text{out}}|} \exp(\ell_r)}. \quad (7.3)$$

于是得到了下一个 token 的概率分布。

这和注意力里的 softmax 是同一种数学运算, 但对象不同: 注意力在 “信息来源的位置” 之间分配权重, 输出层在 “词表候选” 之间分配概率。softmax 本身不负责选中某个 token; 还需要解码规则, 例如取概率最高者, 或按照分布采样。

为展示最简单的流程, 本文采用逐步取最大概率的贪心选择:

$$\hat{y}_t = \arg \max_j P(y_t = j \mid s, \hat{y}_{<t}).$$

这只是最简单的演示选择：训练后的模型未必给正确答案最高概率，贪心也不保证整句概率最大，但用它看清流程已经足够。

7.3 生成循环

假设每一步恰好选出期望的字符，整个过程如表 3 所示。

表 3: 一次生成轨迹（教学演示）

步骤	右边已经知道的输入前缀	新选出的 token
1	<BOS>	阿
2	<BOS> 阿	姆
3	<BOS> 阿姆	斯
4	<BOS> 阿姆斯	特
5	<BOS> 阿姆斯特	朗
6	<BOS> 阿姆斯特朗	<EOS>

选到结束标记 <EOS> 后停止，去掉特殊标记，显示“阿姆斯特朗”。实际也可设置最大生成长度，避免一直不结束。

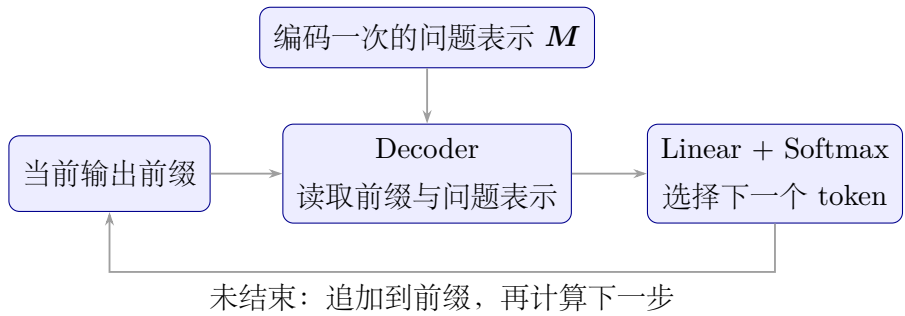


图 6: 生成循环

这解释了“并行计算”和“逐步生成”为什么并不矛盾：对于已知前缀，同一层的各位置可以一起计算；但新的 token 尚未被选出，下一次前缀就还没有确定。实际系统可以缓存已经算过的 Key、Value 来避免部分重复工作，这不改变这里的逻辑。

最后留意一个细节：“阿姆斯特朗”并没有出现在输入问题里，模型照样能输出它——因为最后一层是在整个词表上打分，而不是从输入句子里挑词。至于哪个词该得高分，由训练学到的参数和当前输入共同决定。

7.4 训练时的输入：Outputs shifted right

前面的生成过程默认模型已经训练完毕，最后补上这块拼图：训练到底在优化什么。训练时手里已经有问题和正确答案。原图右下角的“Outputs shifted right”，说的就是怎么把正确答案变成 Decoder 的输入：在开头补 <BOS>，让输入与预测目标错开一位。

表 4: 训练时的输入与目标对齐

位置	1	2	3	4	5	6
Decoder 输入	<BOS>	阿	姆	斯	特	朗
应该预测	阿	姆	斯	特	朗	<EOS>

例如输入“姆”的位置，目标是预测“斯”。虽然训练矩阵中右边也放着“斯”，因果掩码阻止这个位置读取它。模型没有偷看当前要预测的答案。

这类使用真实前缀进行训练的方式常称为 teacher forcing。完整答案已知，使不同目标位置的损失能够在一次前向中一起计算；生成时则没有真实前缀可用，只能追加自己已经选出的 token。

7.5 预测损失

把源句记为 s ，目标序列 $y = (y_1, \dots, y_m)$ 包含结束标记。自回归形式为

$$p_{\theta}(y \mid s) = \prod_{i=1}^m p_{\theta}(y_i \mid s, y_{<i}). \tag{7.4}$$

让训练答案获得更高概率，可以等价地最小化负对数似然：

$$\mathcal{L}(\theta) = -\frac{1}{m} \sum_{i=1}^m \log p_{\theta}(y_i \mid s, y_{<i}). \tag{7.5}$$

这里给出未使用标签平滑、未考虑 padding 的基本形式；实际批量训练需忽略补齐位置，原论文还使用了标签平滑。

如果某一位置正确 token 的预测概率从 0.1 提高到 0.8，这一位置的损失就从 $-\log 0.1 \approx 2.303$ 降到 $-\log 0.8 \approx 0.223$ ——训练自始至终在做的，就是把正确答案的概率推高。

θ 包括输入和输出的嵌入、各头的投影、输出混合矩阵、FFN、LN 的缩放和平移、词表输出层等参数。原论文在适用的共享词表设置中还共享了部分嵌入和输出权重；为了把数据流讲清楚，本文没有要求这些符号必须相互独立。

表 5: 可训练参数与当前输入产生的中间结果

类别	例子	它们如何变化
可训练参数	嵌入表、各类 $\mathbf{W}$ 、偏置、 γ 和 β	根据训练损失更新；正常推理时固定
中间结果	$\mathbf{X}, \mathbf{Q}, \mathbf{K}, \mathbf{V}, \mathbf{A}, \mathbf{Z}, \mathbf{M}$	每次随当前输入和参数重新计算
固定规则	正弦位置编码、因果掩码、选定的模型尺寸	在本文的设定中不是梯度优化的参数

因此，“训练注意力”并不是直接存下一张永不变化的关联表，而是学习产生投影与权重的参数。最终损失沿着完整计算图反向传播，包括注意力、Add、Norm、FFN 和嵌入。没有人提前告诉每一层的向量应该等于什么数值。

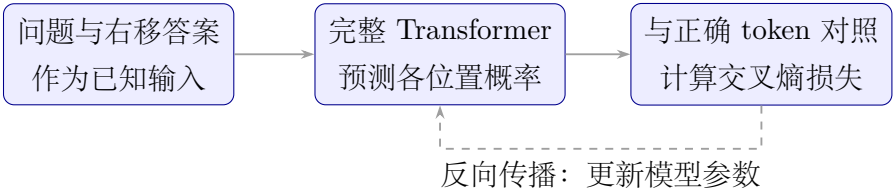


图 7: 训练的数据流

8 小结

到这里，Transformer 的每个部件都拆开看过了。串起来再走一遍：它始终在处理一串向量——嵌入把离散编号变成特征，位置编码引入顺序；注意力根据匹配关系汇总不同位置的信息，多头保留多套汇总方式；残差把模块组织成对已有表示的更新，归一化控制尺度，FFN 在每个位置内部进一步构造非线性特征。

左侧将问题编码成 $\mathbf{M}$ ，右侧根据已有答案前缀构造 Query，通过交叉注意力读取 $\mathbf{M}$ 。最后一个位置的向量被映射到词表分数，再变成下一 token 的概率；选出的 token 被追加到前缀，直到结束。训练则用正确答案的概率定义损失，将这整条链上的参数一起调整。

$$\boxed{\text{构造表示} \rightarrow \text{交换与变换信息} \rightarrow \text{输出条件概率}} \quad (8.1)$$

最值得记住的不是那张图里有多少个框，而是每个框改变了什么。 $\mathbf{A}$ 是信息分配比例，不是最终特征；Add 是模型定义的一部分，不是训练完再补上的修正；LN 接收的是前一步计算的结果，不要求括号里永远只有一个字母；FFN 的表达力来自可学习的非线性变换，而不是单纯的升维。

“Transformer”也并不强制所有模型都必须保留原图中的两座塔。这里讨论的是原始编码器—解码器版本；理解它的模块和数据流以后，再看使用其中部分结构的模型，就可以逐项比较它保留了哪些连接、修改了哪些约束，而不必重新从术语开始。